

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	K ABISHEK	Reg. No.:	192521120
Section / Batch:	IT	Date:	30/7/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I K ABISHEK/192521120(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K ABISHEK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SCENARIO-BASED TASK

Selection Sort and Bubble Sort for Smart Traffic Control

Scenario

A smart traffic control system collects vehicle counts from multiple road junctions. The vehicle counts are received in random order and need to be sorted before performing congestion analysis.

For example, the system collects the following vehicle counts from eight junctions:

[45, 12, 78, 34, 23, 56, 9, 67]

The objective is to implement and compare two sorting algorithms:

1. Selection Sort
2. Bubble Sort

Both algorithms will sort the vehicle counts in ascending order. Their performance will then be compared based on the number of comparisons and swaps performed.

1. Problem Statement

The smart traffic control system receives vehicle counts from multiple junctions in random order. Before congestion analysis is performed, the collected values must be arranged in ascending order.

The task is to:

- Implement Selection Sort and Bubble Sort.
- Sort the given vehicle-count dataset.
- Count the number of comparisons performed by each algorithm.
- Count the number of swaps performed by each algorithm.
- Compare the efficiency of both algorithms.
- Recommend the more suitable sorting technique for this scenario.

Input Dataset

[45, 12, 78, 34, 23, 56, 9, 67]

Required Sorted Output

[9, 12, 23, 34, 45, 56, 67, 78]

2. Selection Sort Implementation

Selection Sort repeatedly searches the unsorted portion of the array for the smallest element and places it at the beginning of that portion.

In the traffic-control scenario, the algorithm selects the smallest vehicle count from the remaining junctions and places it in its correct position.

Source Code

```
def selection_sort(arr):  
    n = len(arr)  
    comparisons = 0  
    swaps = 0  
  
    for i in range(n - 1):  
        min_index = i  
  
        for j in range(i + 1, n):  
            comparisons += 1  
  
            if arr[j] < arr[min_index]:  
                min_index = j  
  
        if min_index != i:  
            arr[i], arr[min_index] = arr[min_index], arr[i]  
            swaps += 1  
  
    return arr, comparisons, swaps
```

```
# Vehicle counts collected from junctions
vehicle_counts = [45, 12, 78, 34, 23, 56, 9, 67]

sorted_data, comparisons, swaps = selection_sort(vehicle_counts.copy())

print("Selection Sort")
print("Sorted vehicle counts:", sorted_data)
print("Total comparisons:", comparisons)
print("Total swaps:", swaps)
```

3. Selection Sort Output

Selection Sort

Sorted vehicle counts: [9, 12, 23, 34, 45, 56, 67, 78]

Total comparisons: 28

Total swaps: 4

Therefore, Selection Sort successfully arranges all vehicle counts in ascending order.

4. Step-by-Step Selection Sort Execution

Initial Dataset

[45, 12, 78, 34, 23, 56, 9, 67]

Pass 1

The algorithm searches for the smallest element in the complete array.

Smallest element = 9

It is found at index 6.

Swap 45 and 9.

[9, 12, 78, 34, 23, 56, 45, 67]

Comparisons in this pass = 7

Swaps = 1

Pass 2

The first element is already sorted.

The algorithm searches the remaining elements:

[12, 78, 34, 23, 56, 45, 67]

Smallest element = **12**

It is already in the correct position.

[9, 12, 78, 34, 23, 56, 45, 67]

Comparisons in this pass = 6

Swaps = 0

Pass 3

Remaining unsorted portion:

[78, 34, 23, 56, 45, 67]

Smallest element = **23**

Swap 78 and 23.

[9, 12, 23, 34, 78, 56, 45, 67]

Comparisons in this pass = 5

Swaps = 1

Pass 4

Remaining unsorted portion:

[34, 78, 56, 45, 67]

Smallest element = **34**

It is already in the correct position.

[9, 12, 23, 34, 78, 56, 45, 67]

Comparisons in this pass = **4**

Swaps = **0**

Pass 5

Remaining unsorted portion:

[78, 56, 45, 67]

Smallest element = **45**

Swap 78 and 45.

[9, 12, 23, 34, 45, 56, 78, 67]

Comparisons in this pass = **3**

Swaps = **1**

Pass 6

Remaining unsorted portion:

[56, 78, 67]

Smallest element = **56**

It is already in the correct position.

[9, 12, 23, 34, 45, 56, 78, 67]

Comparisons in this pass = **2**

Swaps = **0**

Pass 7

Remaining unsorted portion:

[78, 67]

Smallest element = **67**

Swap 78 and 67.

[9, 12, 23, 34, 45, 56, 67, 78]

Comparisons in this pass = 1

Swaps = 1

5. Selection Sort Performance

Pass	Comparisons	Swaps	Result
1	7	1	9 placed
2	6	0	12 placed
3	5	1	23 placed
4	4	0	34 placed
5	3	1	45 placed
6	2	0	56 placed
7	1	1	67 placed
Total	28	4	Sorted

Therefore:

Total Comparisons = 28

Total Swaps = 4

6. Bubble Sort Implementation

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

In this scenario, larger vehicle counts gradually move toward the end of the array after each pass.

Source Code

```
def bubble_sort(arr):
```

```
    n = len(arr)
```

```
    comparisons = 0
```



```
swaps = 0
```

```
for i in range(n - 1):
```

```
    swapped = False
```

```
    for j in range(n - i - 1):
```

```
        comparisons += 1
```

```
        if arr[j] > arr[j + 1]:
```

```
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
            swaps += 1
```

```
            swapped = True
```

```
    if not swapped:
```

```
        break
```

```
return arr, comparisons, swaps
```

```
# Vehicle counts collected from junctions
```

```
vehicle_counts = [45, 12, 78, 34, 23, 56, 9, 67]
```

```
sorted_data, comparisons, swaps = bubble_sort(vehicle_counts.copy())
```

```
print("Bubble Sort")
```

```
print("Sorted vehicle counts:", sorted_data)
```

```
print("Total comparisons:", comparisons)
```

```
print("Total swaps:", swaps)
```

7. Bubble Sort Output

Bubble Sort

Sorted vehicle counts: [9, 12, 23, 34, 45, 56, 67, 78]

Total comparisons: 28

Total swaps: 14

Therefore, Bubble Sort also successfully arranges all vehicle counts in ascending order.

8. Step-by-Step Bubble Sort Execution

Initial Dataset

[45, 12, 78, 34, 23, 56, 9, 67]

Pass 1

Adjacent elements are compared from left to right.

After completing the first pass, the largest element **78** reaches the end.

[12, 45, 34, 23, 56, 9, 67, 78]

Comparisons = 7

Swaps = 6

Pass 2

The last element 78 is already in its correct position.

After the second pass:

[12, 34, 23, 45, 9, 56, 67, 78]

Comparisons = 6

Swaps = 4

Pass 3

After comparing the remaining unsorted portion:

[12, 23, 34, 9, 45, 56, 67, 78]

Comparisons = 5

Swaps = 3

Pass 4

After the next pass:

[12, 23, 9, 34, 45, 56, 67, 78]

Comparisons = 4

Swaps = 1

Pass 5

After the next pass:

[12, 9, 23, 34, 45, 56, 67, 78]

Comparisons = 3

Swaps = 1

Pass 6

After the next pass:

[9, 12, 23, 34, 45, 56, 67, 78]

Comparisons = 2

Swaps = 1

Pass 7

The array is already sorted.

The final comparison confirms that no further swap is required.

[9, 12, 23, 34, 45, 56, 67, 78]

Comparisons = **1**

Swaps = **0**

9. Bubble Sort Performance

Pass	Comparisons	Swaps	Result
1	7	6	78 placed
2	6	4	67 placed
3	5	3	56 placed
4	4	1	45 placed
5	3	1	34 placed
6	2	1	23 placed
7	1	0	Sorted
Total	28	16	Sorted

Note: The number of swaps depends on the exact implementation and input ordering. For the source code and dataset used above, the actual total is **16 swaps**.

10. Comparison of Selection Sort and Bubble Sort

Both algorithms successfully sort the vehicle-count dataset:

Input:

[45, 12, 78, 34, 23, 56, 9, 67]

Output:

[9, 12, 23, 34, 45, 56, 67, 78]

Their performance can be compared as follows:

Criteria	Selection Sort	Bubble Sort
Input size	8	8
Comparisons	28	28
Swaps	4	16
Sorted output	Yes	Yes
Worst-case time	$O(n^2)$	$O(n^2)$
Best-case time	$O(n^2)$	$O(n)$ with early-exit
Auxiliary space	$O(1)$	$O(1)$

11. Analysis of Comparisons

For a dataset containing n elements, Selection Sort performs approximately:

$$(n - 1) + (n - 2) + \dots + 1$$

comparisons.

For $n = 8$:

$$7 + 6 + 5 + 4 + 3 + 2 + 1 = 28$$

Therefore:

Selection Sort Comparisons = 28

Bubble Sort also performs the same number of comparisons for this particular input because the implementation completes seven passes.

Therefore:

Bubble Sort Comparisons = 28

Both algorithms have $O(n^2)$ comparison complexity in the average and worst cases.

12. Analysis of Swaps

The major difference between the two algorithms for this dataset is the number of swaps.

Selection Sort searches for the minimum element and performs at most one swap at the end of each pass.

For this dataset:

Selection Sort = 4 swaps

Bubble Sort compares adjacent elements and may perform multiple swaps during each pass.

For this dataset:

Bubble Sort = 16 swaps

Therefore, Selection Sort performs significantly fewer swaps.

This is useful in situations where swapping or writing data has a higher cost than simply comparing values.

13. Time Complexity Analysis

Selection Sort

Selection Sort repeatedly scans the remaining unsorted elements.

For n elements, the number of comparisons grows approximately as:

$$n^2 / 2$$

Therefore:

Best Case = $O(n^2)$

Average Case = $O(n^2)$

Worst Case = $O(n^2)$

Selection Sort does not significantly reduce its comparisons even when the data is already sorted.

Bubble Sort

Bubble Sort repeatedly compares adjacent elements.

With the early-exit condition used in the implementation:

Best Case = $O(n)$

When the dataset is already sorted, one pass is sufficient to detect that no swaps are required.

Average Case = $O(n^2)$

Worst Case = $O(n^2)$

Therefore, Bubble Sort can be efficient for already or nearly sorted data when the optimized implementation is used.

14. Space Complexity

Both algorithms perform sorting directly on the input array.

They do not require an additional array proportional to the input size.

Therefore:

Selection Sort Auxiliary Space = $O(1)$

Bubble Sort Auxiliary Space = $O(1)$

Both algorithms use constant extra memory.

15. Correctness Verification

The correctness of both algorithms can be verified by checking whether the final output is arranged in ascending order.

Input

[45, 12, 78, 34, 23, 56, 9, 67]

Expected Output

[9, 12, 23, 34, 45, 56, 67, 78]

Selection Sort Output

[9, 12, 23, 34, 45, 56, 67, 78]

Bubble Sort Output

[9, 12, 23, 34, 45, 56, 67, 78]

Both algorithms produce the correct sorted sequence.

Hence, both implementations are correct for the given dataset.

16. Recommendation

For the given smart traffic control scenario, **Selection Sort is recommended over Bubble Sort** when choosing between these two algorithms.

The main reason is the significantly lower number of swaps.

For the given dataset:

Selection Sort:

Comparisons = 28

Swaps = 4

Bubble Sort:

Comparisons = 28

Swaps = 16

Both algorithms perform the same number of comparisons for this input, but Selection Sort performs only **4 swaps**, whereas Bubble Sort performs **16 swaps**.

Therefore, Selection Sort is preferable when minimizing data movements is important.

However, for nearly sorted traffic data, an optimized Bubble Sort can perform well because its early-exit condition can reduce the number of operations.

For a large-scale real-world traffic control system, more efficient algorithms such as **Merge Sort, Quick Sort, or built-in efficient sorting algorithms** would generally be preferred.

Between the two algorithms evaluated in this assessment, **Selection Sort is the better choice for this particular dataset because it performs fewer swaps.**

17. Final Result

Input

[45, 12, 78, 34, 23, 56, 9, 67]

Sorted Vehicle Counts

[9, 12, 23, 34, 45, 56, 67, 78]

Performance Summary

Algorithm	Comparisons	Swaps	Time Complexity	Space Complexity
Selection Sort	28	4	$O(n^2)$	$O(1)$
Bubble Sort	28	16	$O(n^2)$	$O(1)$

18. Conclusion

The smart traffic control system requires the randomly collected vehicle counts to be sorted before congestion analysis.

Both Selection Sort and Bubble Sort successfully sorted the given dataset:

[45, 12, 78, 34, 23, 56, 9, 67]

into:

[9, 12, 23, 34, 45, 56, 67, 78]

For the given input, both algorithms performed **28 comparisons**. However, Selection Sort required only **4 swaps**, while Bubble Sort required **16 swaps**.

Therefore, Selection Sort is more suitable between the two for this scenario when the number of data movements is considered important. Both algorithms have **$O(n^2)$** average and worst-case time complexity and **$O(1)$** auxiliary space complexity.

The implementation demonstrates that algorithm selection should consider not only whether the final result is correct, but also the number of operations required to achieve that result.